# Memory Fragmentation Simulator

An interactive, offline tool that shows how contiguous memory becomes fragmented as processes are allocated and freed, and how coalescing, compaction and the buddy system deal with it.

## How to run

1. Double-click `index.html`. It opens in Chrome (or any modern browser).
2. That's it. The page is a single file with HTML, CSS and vanilla JavaScript. It uses no frameworks, no libraries and no internet connection. Keyboard shortcuts (when the cursor is not in a text box): → next step, ← previous step, **Space** play/pause.

## Features

| Area | What you can do |
|---|---|
| Memory model | Contiguous memory of configurable size (default 1024 KB), drawn as a horizontal bar. Processes are coloured blocks labelled with name and size. Holes are grey with their size. Hover any block for its exact addresses. |
| Manual actions | Allocate (name + size), free (click a block, use the dropdown, or the **Free** button in the table), compact, reset, set the total size. |
| Allocation strategies | First Fit, Best Fit, Worst Fit, Next Fit (the next-fit pointer is drawn as ▼). |
| Failure explanations | Every failed allocation says *why*, for example: `Failed (P7): 150 KB requested, 324 KB free in total, but the largest hole is only 124 KB → external fragmentation. Compaction would fix this.` It tells apart external fragmentation, plain out-of-memory, and requests bigger than memory. It also notes when coalescing would have helped, or when internal fragmentation is wasting the needed space. |
| External fragmentation stats | Number of holes, total free, largest hole, and `ext. frag = 1 − largest hole / total free`. |
| Internal fragmentation | *Fixed block size* mode rounds each request up to a multiple of the block size (default 64 KB). The waste is drawn hatched inside each block and totalled. |
| Coalescing | A toggle. With it OFF, freed blocks stay separate holes even when they sit side by side. **Merge holes now** merges them on demand. |
| Compaction | An animated, one process at a time slide towards address 0. It reports how many KB had to be copied (the cost). |
| Buddy system | Power-of-2 splitting and buddy merging (`buddy = address XOR size`). |
| Workloads | A seeded random workload (N, seed, free %, size range), so the same seed always gives the same requests. Four scripted demos are also included. |
| Step-by-step mode | Next / Previous / Start / Auto-play with a speed slider. Previous truly undoes, because every step is snapshotted. |
| Comparison | Runs the SAME workload through First/Best/Worst/Next Fit, plus the buddy system as a reference. The table shows allocations, failures, failures caused by external fragmentation, final free memory, holes, largest hole, final and average fragmentation, and internal fragmentation. It also draws a mini memory map for each strategy. |
| Event log | Every action and its result, with the step number. |
| Explanations | A side panel in plain English. The current strategy is highlighted. |

### Built-in demos (good for the viva)

| Demo | What it shows |
|---|---|
| External fragmentation → compaction | P1–P6 use 900 KB. Freeing P2 and P4 leaves holes of 100 + 100 + 124 KB. **Step 9:** P7 (150 KB) fails with 324 KB free (61.7% external fragmentation). **Step 10:** compaction moves 500 KB. **Step 11:** P7 fits. |
| Coalescing OFF vs ON | Runs with coalescing OFF: P4 (500 KB) fails even though 724 KB of free space is contiguous. Tick *Coalesce*, press ⏮, replay: it succeeds. |
| Internal fragmentation | With 64 KB blocks, 764 KB of requests take 960 KB of memory (196 KB wasted), so P6 fails. |
| Buddy system | Shows splits (1024 → 512 → 256 → 128). P5 needs 512 KB and fails while two 256 KB blocks are free, because they are not buddies. Once P2 is freed they merge and P5 fits. |

## How each algorithm works

Memory is stored as a list of blocks sorted by address, with no gaps: `{ start, size, name, requested }`. `name = null` means the block is a hole. All algorithms live in the `MemoryManager` class (section 2 of the script), which contains no UI code. That is why the comparison view can run four copies side by side.

### Allocation strategies (`chooseHole`)

- **First Fit:** walk the holes in address order and return the first one with `size ≥ request`.
- **Best Fit:** look at every hole and keep the smallest that is still big enough.
- **Worst Fit:** look at every hole and keep the largest.
- **Next Fit:** keep `nextPtr`, the address where the previous allocation ended. Start searching at the hole containing or after `nextPtr`, wrap around once, and take the first fit.

The chosen hole is **split**: the process takes the front part and the remainder (if any) stays a smaller hole.

### Failure classification (`fail`)

1. `need > total memory`: the request is too large for the whole memory.
2. `total free < need`: out of memory. This is **not** fragmentation. If the internal waste would have covered the shortfall, the message says so.
3. `total free ≥ need`, but no single hole is that big: **external fragmentation**. If adjacent holes together would fit, the message says that coalescing would have fixed it. Otherwise it says that compaction would.

### Fixed block size (`roundUp`)

`allocated = ceil(request / blockSize) × blockSize`. The internal fragmentation of a process is `allocated − requested`.

### Coalescing (`coalesceAround`)

When a block is freed, it becomes a hole. While the block to its right is a hole, the two are merged. While the block to its left is a hole, the two are merged. Only the neighbours are checked, so the cost is O(1) merges per free.

### Compaction (`compactStep`)

Keep a `cursor` starting at 0. Walk the processes in address order. If a process does not start at `cursor`, move it there and add its size to "KB moved". Then `cursor += size`. Afterwards every gap is rebuilt, which leaves exactly one hole at the end. The UI calls `compactStep()` once per tick, so you see each move.

### Buddy system (`allocateBuddy`, `mergeBuddies`)

- **Allocate:** round up to `max(minBlock, nextPowerOf2(request))`. Take the smallest free block that is big enough. While it is too big, split it into two halves (buddies) and keep the left half.
- **Free:** the buddy of a block at address `A` with size `S` is at `A XOR S`. If that buddy is free and has the same size, merge the two into a block of size `2S`. Repeat until the buddy is busy or the whole memory is one block.

### Seeded random workload (`mulberry32`, `generateRandomWorkload`)

Mulberry32 is a tiny pseudo-random number generator: the same seed always produces the same numbers. At each step, the generator frees a random "live" process with probability *free %*. Otherwise it allocates a new process `Pk` with a random size. If an allocation fails during a run, the later FREE of that process is logged as "nothing to free". That happens identically for every strategy, so the comparison stays fair.

### Step-by-step undo

After every step the player stores a deep copy of the memory state: `history[i]` is the state after `i` steps. **Previous** restores `history[i-1]` and trims the log back to its length at that point.

### Comparison (`runHeadless`)

For each strategy the comparison creates a fresh `MemoryManager` with the same settings and replays the identical operation list. After each step it records the fragmentation metric, which gives the average column. At the end it reports the counters and final stats.

## Code map (inside `index.html`)

1 Utilities
2 `MemoryManager`: every OS algorithm
3 Workloads: PRNG, random generator, demo scenarios
4 Application state
5 Rendering (memory bar, stats, tables, log)
6 User actions (allocate, free, animated compaction)
7 Step-by-step player
8 Comparison
9 Event wiring + start-up

---

## 10 likely viva questions (with short answers)

**1. What is the difference between internal and external fragmentation?**
Internal fragmentation is unused space *inside* an allocated block. It happens because allocation units (blocks, pages, buddy sizes) are bigger than the request. External fragmentation is free space *between* allocated blocks. Total free memory is enough, but it is not contiguous, so a large request cannot be placed.

**2. How do you measure external fragmentation here, and why that formula?**
`1 - largest hole / total free memory`. If all free memory is one hole, the result is 0%: any request up to the free total fits. As free memory splits into many small holes, the largest hole shrinks relative to the total and the value approaches 100%. It measures how much of the free memory is unusable for one big request.

**3. Compare First, Best, Worst and Next Fit.**
First Fit takes the first hole that fits, which is fast. Best Fit takes the smallest hole that fits; it leaves tiny slivers and must search the whole list. Worst Fit takes the largest hole and quickly destroys big holes. Next Fit is First Fit that resumes from the last allocation. Simulations (Silberschatz) show First Fit and Best Fit beat Worst Fit in storage use, and First Fit is usually faster.

**4. Why can an allocation fail even when enough memory is free?**
Contiguous allocation needs one hole at least as large as the request. With external fragmentation the free memory is split into holes that are each too small, for example 324 KB free but a largest hole of only 124 KB for a 150 KB request.

**5. What is coalescing and how is it different from compaction?**
Coalescing merges a freed block with *adjacent* free holes. It is cheap and happens on every free, but it cannot join holes that have an allocated block between them. Compaction *moves* allocated blocks so that all holes join into one. It removes external fragmentation completely but is expensive.

**6. What does compaction cost, and when is it possible?**
Every byte of every moved process has to be copied, and those processes are paused (the simulator reports the KB moved). Compaction needs dynamic relocation, meaning execution-time address binding with a base/relocation register. With compile-time or load-time binding, the process's addresses can't change, so it can't be moved.

**7. How does the buddy system work, and how do you find a block's buddy?**
Memory is a power of 2 and requests are rounded up to a power of 2. Larger blocks are split in halves until the size matches. On free, a block merges with its buddy if the buddy is free, and this repeats upward. The buddy of a block at address A with size S is at **A XOR S**.

**8. What are the drawbacks of the buddy system?**
Internal fragmentation of up to about 50% (129 KB needs 256 KB). Two adjacent free blocks that are *not* buddies cannot merge, so some external fragmentation remains. Its advantage is very fast splitting and merging; Linux uses it for physical page frames.

**9. What is the 50-percent rule?**
Knuth's statistical result for First Fit: given N allocated blocks, about 0.5 N blocks are lost to fragmentation. That means up to one-third of memory may be unusable.

**10. How do paging and segmentation relate to fragmentation?**
Paging lets a process use non-contiguous frames, so it **eliminates external fragmentation**. It still has internal fragmentation, on average half a page in the last page. Segmentation uses variable-size segments, so it has **external** fragmentation and needs the same placement strategies and compaction shown here. Fixed block size mode in this simulator is a step towards paging, but each process here still has to be contiguous.